

ภาควิชาคณิตศาสตร์ คณะวิทยาศาสตร์ประยุกต์ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

เอกสารประกอบการสอน

040223101 การเขียนโปรแกรมคอมพิวเตอร์สำหรับคณิตศาสตร์ 1

(Computer Programming for Mathematics 1)

บทที่ 4

โครงสร้างการตัดสินใจ

(Conditional Structure)

ผู้สอน: รศ.ดร.อนุชิต จิตพัฒนกุล

ภาคการศึกษาที่ 1 ปีการศึกษา 2569

วัตถุประสงค์การเรียนรู้ประจำบท

เมื่อศึกษาบทนี้จบ นักศึกษาสามารถ

- ใช้ตัวดำเนินการเปรียบเทียบและตัวดำเนินการเชิงตรรกะสร้างนิพจน์เงื่อนไขได้
- เขียนคำสั่ง if, if-else และ if-elif-else เพื่อควบคุมการทำงานของโปรแกรมได้
- อธิบายความสำคัญของการเยื้อง (indentation) ในการกำหนดบล็อกคำสั่งได้
- เขียนเงื่อนไขซ้อน (nested if) เพื่อแก้ปัญหาที่มีหลายระดับได้
- ประยุกต์โครงสร้างการตัดสินใจแก้โจทย์ปัญหาทางคณิตศาสตร์ได้

CLO: เนื้อหาบทนี้สอดคล้องกับผลลัพธ์การเรียนรู้ของรายวิชา (CLO 3)

บทที่ 4 โครงสร้างการตัดสินใจ

จนถึงตอนนี้โปรแกรมของเราทำงานเรียงลำดับจากบนลงล่างทีละบรรทัด แต่ปัญหาในชีวิตจริงมักต้อง "ตัดสินใจ" เลือกทำสิ่งต่างๆ ตามเงื่อนไข เช่น ถ้าคะแนนถึงเกณฑ์จึงผ่าน บทนี้จะศึกษาโครงสร้าง การตัดสินใจซึ่งทำให้โปรแกรมเลือกเส้นทางการทำงานได้ตามเงื่อนไขที่กำหนด

4.1 นิพจน์เงื่อนไขและตัวดำเนินการเปรียบเทียบ

นิพจน์เงื่อนไข (condition) คือนิพจน์ที่ให้ผลลัพธ์เป็นค่าตรรกะ True หรือ False ตัวดำเนินการ เปรียบเทียบที่ใช้สร้างเงื่อนไขมีดังนี้

ตัวดำเนินการ	ความหมาย	ตัวอย่าง	ผลลัพธ์
==	เท่ากับ	5 == 5	True
!=	ไม่เท่ากับ	5 != 3	True
>	มากกว่า	5 > 8	False
<	น้อยกว่า	5 < 8	True
>=	มากกว่าหรือเท่ากับ	5 >= 5	True
<=	น้อยกว่าหรือเท่ากับ	3 <= 2	False

ข้อควรระวัง: ระวังสับสนระหว่าง = (กำหนดค่า) กับ == (เปรียบเทียบว่าเท่ากัน) ทั้งสองมีความหมายต่างกันโดยสิ้นเชิง

4.2 ตัวดำเนินการเชิงตรรกะ

เมื่อต้องรวมหลายเงื่อนไขเข้าด้วยกัน ใช้ตัวดำเนินการเชิงตรรกะ ได้แก่ **and** (จริงทั้งคู่จึงจริง), **or** (จริงอย่างน้อยหนึ่งก็จริง) และ **not** (กลับค่าตรรกะ)

ไฟล์ 04logical.py

```
age = 20
income = 15000
print(age >= 18 and income >= 10000)
print(age < 15 or income > 30000)
print(not (age >= 18))
```

ผลลัพธ์:

```
True
False
False
```

4.3 คำสั่ง if และการเยื้อง (indentation)

คำสั่ง `if` จะทำคำสั่งภายในบล็อกก็ต่อเมื่อเงื่อนไขเป็นจริง รูปแบบคือ `if` เงื่อนไข: ตามด้วยบล็อก คำสั่งที่ **เยื้องเข้าไป (indent)** โดยทั่วไปใช้ 4 ช่องว่าง การเยื้องเป็นสิ่งบังคับใน Python เพราะใช้ บอกขอบเขตของบล็อกคำสั่ง (ต่างจากภาษาอื่นที่ใช้วงเล็บปีกกา)

ไฟล์ 04if.py

```
score = 75
if score >= 50:
    print("ผ่านเกณฑ์")
    print("ยินดีด้วย")
print("จบโปรแกรม")
```

ผลลัพธ์:

```
ผ่านเกณฑ์
ยินดีด้วย
จบโปรแกรม
```

สังเกตว่า "จบโปรแกรม" ไม่ได้เยื้อง จึงอยู่นอกบล็อก `if` และทำงานเสมอ

4.4 คำสั่ง if-else

เมื่อต้องการให้โปรแกรมเลือกทำอย่างใดอย่างหนึ่งจากสองทาง ใช้ `else` ระบุสิ่งที่ทำเมื่อเงื่อนไขเป็นเท็จ

ไฟล์ 04ifelse.py

```
num = int(input("ใส่จำนวนเต็ม: "))
if num % 2 == 0:
    print(f"{num} เป็นจำนวนคู่")
else:
    print(f"{num} เป็นจำนวนคี่")
```

ผลลัพธ์:

```
ใส่จำนวนเต็ม: 7
7 เป็นจำนวนคี่
```

4.5 คำสั่ง if-elif-else

เมื่อมีทางเลือกมากกว่าสองทาง ใช้ **elif** (ย่อจาก else if) ตรวจสอบเงื่อนไขเพิ่มเติมเป็นลำดับ Python จะตรวจจากบนลงล่าง และทำเฉพาะบล็อกแรกที่เงื่อนไขเป็นจริง

ไฟล์ 04grade.py

```
score = int(input("ใส่คะแนน: "))
if score >= 80:
    grade = "A"
elif score >= 70:
    grade = "B"
elif score >= 60:
    grade = "C"
elif score >= 50:
    grade = "D"
else:
    grade = "F"
print(f"เกรดที่ได้คือ {grade}")
```

ผลลัพธ์:

```
ใส่คะแนน: 73
เกรดที่ได้คือ B
```

4.6 เงื่อนไขซ้อน (Nested if)

เราสามารถวางคำสั่ง if ไว้ภายในบล็อกของ if อีกชั้นหนึ่งได้ เรียกว่าเงื่อนไขซ้อน ใช้กับปัญหาที่ต้อง ตรวจสอบหลายระดับ ตัวอย่างต่อไปนี้ตรวจสอบว่าจำนวนเป็นบวก ลบ หรือศูนย์ และถ้าเป็นบวกตรวจสอบต่อว่า เป็นคู่หรือคี่

ไฟล์ 04nested.py

```
n = int(input("ใส่จำนวนเต็ม: "))
if n > 0:
    print("จำนวนบวก")
    if n % 2 == 0:
        print("และเป็นจำนวนคู่")
    else:
        print("และเป็นจำนวนคี่")
elif n < 0:
    print("จำนวนลบ")
else:
    print("ศูนย์")
```

ผลลัพธ์:

```
ใส่จำนวนเต็ม: 8
จำนวนบวก
และเป็นจำนวนคู่
```

4.7 ตัวอย่างประยุกต์: ตรวจสอบประเภทของสามเหลี่ยม

รับความยาวด้านสามด้าน ตรวจสอบว่าประกอบเป็นสามเหลี่ยมได้หรือไม่ และจัดประเภทตามด้าน

ไฟล์ 04triangle.py

```
a = float(input("ด้าน a: "))
b = float(input("ด้าน b: "))
c = float(input("ด้าน c: "))
if a + b > c and b + c > a and a + c > b:
    if a == b == c:
        print("สามเหลี่ยมด้านเท่า")
    elif a == b or b == c or a == c:
        print("สามเหลี่ยมหน้าจั่ว")
    else:
        print("สามเหลี่ยมด้านไม่เท่า")
else:
    print("ไม่สามารถประกอบเป็นสามเหลี่ยมได้")
```

ผลลัพธ์:

```
ด้าน a: 5
ด้าน b: 5
ด้าน c: 8
สามเหลี่ยมหน้าจั่ว
```

4.8 สรุปท้ายบท

โครงสร้างการตัดสินใจทำให้โปรแกรมเลือกเส้นทางการทำงานตามเงื่อนไข นิพจน์เงื่อนไขสร้างจากตัวดำเนินการเปรียบเทียบ (==, !=, >, <, >=, <=) และตัวดำเนินการเชิงตรรกะ (and, or, not) คำสั่ง if ทำงานเมื่อเงื่อนไขจริง else ระบุทางเลือกอื่น และ elif เพิ่มทางเลือกหลายระดับ การเยื้องบล็อกคำสั่ง เป็นสิ่งบังคับและมีความสำคัญอย่างยิ่งใน Python

แบบฝึกหัดท้ายบท

1. จงเขียนโปรแกรมรับเลขจำนวนหนึ่ง แล้วแสดงว่าเป็นจำนวนบวก จำนวนลบ หรือศูนย์
2. จงเขียนโปรแกรมรับปี ค.ศ. แล้วตรวจสอบว่าเป็นปีอธิกสุรทินหรือไม่ (หาร 4 ลงตัวแต่ถ้าหาร 100 ลงตัว ต้องหาร 400 ลงตัวด้วย)
3. จงเขียนโปรแกรมรับเลขสามจำนวน แล้วแสดงค่าที่มากที่สุด
4. จงเขียนโปรแกรมรับค่าดัชนีมวลกาย (BMI) แล้วแสดงเกณฑ์ (น้อยกว่า 18.5 ผอม, 18.5–24.9 ปกติ, 25–29.9 น้ำหนักเกิน, ตั้งแต่ 30 ขึ้นไป อ้วน)

เฉลยแบบฝึกหัดบทที่ 4

ข้อ 2 ป้อธิกรรทึน

```
y = int(input("ใส่ปี ค.ศ.: "))
if (y % 4 == 0 and y % 100 != 0) or (y % 400 == 0):
    print(f"{y} เป็นปีอธิกรรทึน")
else:
    print(f"{y} ไม่เป็นปีอธิกรรทึน")
```

ผลลัพธ์:

```
ใส่ปี ค.ศ.: 2024
2024 เป็นปีอธิกรรทึน
```

ข้อ 3 หาค่ามากที่สุด

```
a = float(input("a: "))
b = float(input("b: "))
c = float(input("c: "))
big = a
if b > big:
    big = b
if c > big:
    big = c
print(f"ค่ามากที่สุดคือ {big}")
```

ผลลัพธ์:

```
a: 4
b: 9
c: 6
ค่ามากที่สุดคือ 9.0
```